Packages

- Packages are used to group the classes and interface which have similar functionalities.
- A package is a namespace that organizes a set of related classes, interface, enum and annotation.

Type of Package:

I. Custom Package
II. Built-In Package

I. Custom Package :

- The packages which are developed by the developer as per application or project requirement are called as Custom package or User Define package.
- Package declaration statement must be the first statement in the source file.
  (Refer page C-103).

II. Built-In Package :

- The packages which are already developed and provided along with java and other technologies are called as Built-in packages or Pre-Define Packages.
- You can just import built-in packages and use various functionalities directly.
- Following are the commonly used built-in packages:
  1. java.lang
  2. java.reflect
  3. java.util
  4. java.io
  5. java.sql
  6. java.math
  7. java.awt.
  8. java.nio (from java7)
  9. javax.sql

  etc.

- An application programming interface (API) is a collection of predefined packages, classes and interfaces with their methods, fields and constructors.
- All java API package are prefixed with java or javax.
- Core packages of java starts with java or javax.
- Advanced packages of java starts with javax.
- You can verify the member signature of any class or interface

  First set the path of java bin then

  Javap <fully Qualified Name>

  Example: -

  javap java.lang.Object //valid

  javap Object // invalid

1. java.lang Package

- java.lang package contains the classes and interface which are required for basic java language functionalities.
- java.lang package is by default imported in all the java programs i.e. you can access all the classes and interface of java.lang package in your java programs without importing.

Classes used from java.lang package.

Class	Object	String	String Buffer
StringBuilder(java5)	Boolean	Character	Number
Byte	Short	Integer	Long
Float	Double	Void	Math
Process	System	Runtime	Throwable
Exception	Error	Thread	ThreadGroup
ClassLoader		Etc..	

Interface used from java.lang package.

Cloneable	Comparable	CharSequence
AutoCloseable (java 7)	Runnable	Etc..

1.1. java.lang Object class

- Object is predefined class in java.lang package.
- Object is default super class for all the java classes i.e. you can refer all the member of Object class in your program without extending.
- When one class is define without any extends keyword the Object will be the direct super class otherwise Object will be indirect super class.
- Object class does not have any super class.
- The members available in this class can be referred with any type of reference like class or interface or array etc.
- Object class reference variable can hold any type of object.

Member of javap java.lang.Object class

```
public class java.lang.Object {
 public Object();
 public final native Class getClass();
 public native int hashCode();
 public String toString();
 public boolean equals(Object);
 protected native Object clone();
          throws CloneNotSupportedException
 protected void finalize(); throws Throwable
```

```
public final native void notify();
public final native void notifyAll();
public final native void wait()
          throws InterruptedException;
public final void wait(long, int);
          throws InterruptedException
public final void wait(long);
          throws InterruptedException
}
```

Exploring Method of Object class

Public final native Class getClass()

- When JVM loads the class in main memory then it will create one default object of the type java.lang.getClass.
- You can access complete information about your class by using that default object of type java.lang.Class.
- getClass () method can be used to access the default object created by JVM at the time of loading.

JLANG1.java

```
public class JLANG1 {
public static void main(String[] args){
showClassInfo("SAI");
Student st=new Student();
showClassInfo(st);
Object ob =new Object();
showClassInfo(ob);
}
static void showClassInfo(Object ob){
Class cls=ob.getClass();
System.out.println("Class Name : "+cls.getName());
Class scls=cls.getSuperclass();
if(scls!=null)
System.out.println("Super class : "+getName());
else
System.out.println("No Super Class");
    }
}
class Person{ }
class Student extends Person{ }
```

Public native int hashCode()

- When you create an object then JVM assigns some integer value for that object. This integer value is called as Hass Code value of the object.
- hashCode() method can be used to access Hash Code value of an object.
- The Hash Code value will be used to identify the object uniquely.
- You can override hashCode() method in your class to provide your own implementation.

JLANG2.java

```
public class Jlang2 {
public static void main(String[] args){
System.out.println("** with Student  ** ");
Student st1=new Student(99,96543210);
Student st2=new Student(99,96543210);
Student st3=new Student(88,56789020);
Student st4=st1;

System.out.println(st1.hashCode());
System.out.println(st2.hashCode());
System.out.println(st3.hashCode());
System.out.println(st4.hashCode());

System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st1==st4);
System.out.println(st3==st4);
  }}
class Student {
int sid;
long phone;
Student(int sid, long phone){
this.sid=sid;
this.phone=phone;
  }
/*      public int hashCode() {
            return(int)(phone | sid);
}*/
}
```

**Here, JVM returns a hash code value for the object and Hash code values of st1 and st4 is equal so st1== st4 is showing true. But In last line of code will execute then hash code will override and it will give same hash code for st1, st2 and st4 but compression result is same.

Public String toString()

- When you print reference variable then following tasks will happen:
  - ⇒ If reference variable contains null then null value will be displayed.
  - ⇒ If reference variable contains address of an object then toString() method will be invoked by the JVM automatically.
  - ⇒ By default toString() of object class will print:
    <Class Name>@<Hexadecimal Of HashCode>
- You can override this method in your class to display some meaningful String.
- Usually toString() method is used to print contents of an object . This method is already overridden in many java build-in classes like String, StringBuffer, Integer etc.

JLANG3.java

```
public class JLANG3 {
public static void main(String[] args){
Student st1=null;
System.out.println(st1);
st1=new Student(99 , "SAI");
System.out.println(st1);
String str=new String ("Hello");
System.out.println(str);
Integer ref=new Integer(123);
System.out.println(ref);
    }
}
class Student {
int sid;
String name;
Student(int sid,String name){
this.sid=sid;
this.name=name;
    }
}
```

JLANG4.java

```
public class JLANG4 {
public static void main(String[] args){
Student st1=new Student(99 , "SAI");
System.out.println(st1);
System.out.println("**Def- Implementation**");
String cname=st1.getClass().getName();
int hc=st1.hashCode();
String hx=Integer. toHexString(hc);
String msg=cname+"@"+hx;
System.out.println(msg);
    }
}
class Student {
int sid;
String name;
Student(int sid,String name){
this.sid=sid;
this.name=name;
    }
}
```

JLANG5.java

```
public class JLANG5 {
public static void main(String[] args){
Student st1=null;
System.out.println(st1);
Student st1=new Student (99 , "SAI");
Student st2=new Student (88 , "Ram");
System.out.println(st1);
System.out.println(st2);
    }
}
```

```
class Student {
int sid;
String name;
Student(int sid,String name){
this.sid=sid;
this.name=name;
}
public String toString(){
return sid +"\t"+name;
   }
}
```

**Here,

Public boolean equals (Object obj)

- You can use == Operator to compare two variable of primitive type or reference type. mm
- When you compare primitive type variable using == operator then it will compare the values in primitive variable.
- When you compare reference type variable using == operator then it will compare the address available in reference variables.
- When you want to compare the contents using of two objects that you can use equals () method.
- You can use following versions of equals() method:
  - ⇒ Object class equals () method.
  - ⇒ Overridden equals () method.
- Object class equals () method will always compares the address of two objects.

```
public boolean equals (Object obj){
return this == obj;
}
```

- You can override equals () method in your class when you want to compare contents of two objects.
- Usually equals () method is overridden in many java built-in class to compare contents of objects.

JLANG6.java

```
public class JLANG6 {
public static void main(String[] args){
String st1=new String("SAI");
String st2=new String("SAI");
String st3=new String("RAM");
System.out.println("Using == operator");
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println("Using equals () ");
System.out.println(st1.equals(st2));
System.out.println(st1.equals(st3));
  }
}
```

JLANG7.java

```
public class JLANG7 {
public static void main(String[] args){
Student st1=new Student(99, "SAI");
Student st2=new Student(99, "SAI");
Student st3=new Student(88, "RAM");
Student st4=st1;

System.out.println("Using == operator");
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st1==st4);
System.out.println(st2==st3);
System.out.println("Using equals () ");
System.out.println(st1.equals(st2));
System.out.println(st1.equals(st3));
System.out.println(st1.equals(st4));
System.out.println(st2.equals(st3));
  }
}
class Student{
   int sid;
   String name;
 Student(int sid, String name) {
      this .sid=sid;
      this.name=name;
   }
}
```

**Here,

JLANG8.java

```
public class JLANG7 {
public static void main(String[] args){
Student st1=new Student(99, "SAI");
Student st2=new Student(99, "SAI");
Student st3=new Student(88, "RAM");
Student st4=st1;
System.out.println("Using == operator");
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st1==st4);
System.out.println(st2==st3);
System.out.println("Using equals () ");
System.out.println(st1.equals(st2));
System.out.println(st1.equals(st3));
System.out.println(st1.equals(st4));
System.out.println(st2.equals(st3));
 } }
```

```
class Student{
   int sid;
   String name;
  Student(int sid, String name) {
      this .sid=sid;
      this.name=name;
   }
public boolean equals(Object obj){
if (obj instanceof Student){
Student st=(Student)obj;
return this.sid==st.sid && this.name. equals
(st. name);
}
return false;
}
}
```

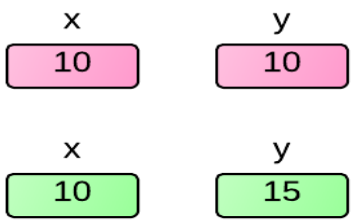
** Here:-

Cloning

- When you want to create the copy of primitive variable then you can use = operator.
- After creating the copy of primitive variable, if you do some modification on copy variable then original value will not be affected.

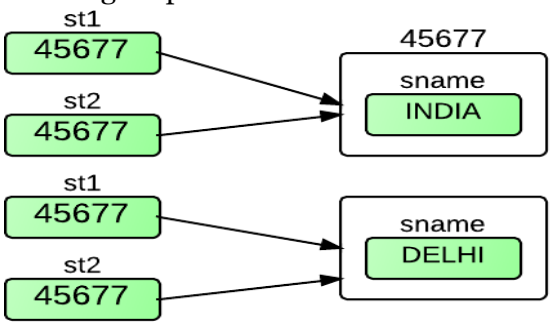
```
int x=10;
// creating copy
   int y=x;

//Modifying using copy
   y=y+5
```



- When you are using = operator to create the copy of reference variable then address of existing object will be copied to the reference variable.
- When you do some modification with object data using copy reference then original object will be modified because both the reference variable is pointing to same object.
- So we cannot create copy of object variable using = operator.

```
class Student {
 String name;
}
Student st1=new Student();
    st1.name= “INDIA”;
//creating  copy
    Student st2=st1;
//Modifying data using copy
   st2.name= “Bihar”;
```



- You can use cloning to create the copy of object.
- Cloning is the process of creating copy of existing object.

Protected native Object clone() throws CloneNotSupportedException

- You can create the copy of existing object by invoking clone () method of Object class.
- clone() method is define as protected in object class. So outside the java.lang package you can access in the subclass directly or with that subclass object only.
- When you want to clone any class object then you must override clone () method in the class.
- Java class object are not eligible for cloning by default.
- Cloneable marker interface is used to instruct the JVM that particular class object is eligible for cloning.
- When you want to clone any class object then that class has to implement marker interface called java.lang. Cloneable.
- When you use clone() method of Object class then the constructor of your class will not be used to create the new object.
- When you use the clone () method of object class then it is mandatory to implement Cloneable interface.
- clone() method is throwing checked exception called CloneNotSupportedException.
- When you clone any class object without implementing Cloneable interface then CloneNotSupportedException will be thrown by the JVM.
- When you override the clone() method then you have two options.

Case 1: You can force to implement Cloneable interface.
Case 2: You can ignore the implementation of cloneable interface.

Case 1: You can force to implement Cloneable interface.

```
public Object clone()throws CloneNotSupportedException{
        if (this instanceof  Cloneable){
        Hai hai=new Hai(this.hai.x);
        Hello h1=new Hello(this.y,hai);
        return h;
    }
else
   throw new CloneNotSupportedException (getClass().getName());
}
```

Case 2: You can ignore the implementation of cloneable interface.

```
public Object clone()throws CloneNotSupportedException{
        Hai hai=new Hai(this.hai.x);
        Hello h1=new Hello(this.y,hai);
        return h;
    }
```

Type of Cloning

1. Shallow Cloning
2. Deep Cloning

1. Shallow Cloning

- This is the default implementation of java.lang.Object class clone() method.
- In this type of cloning the current object (that is used to invoke clone) will be cloned.
- If any member objects available then that will not be cloned.
- If you modify the data of clone object then it won't affect the actual object.
- If you modify the data of the member object then it will affect the other object also

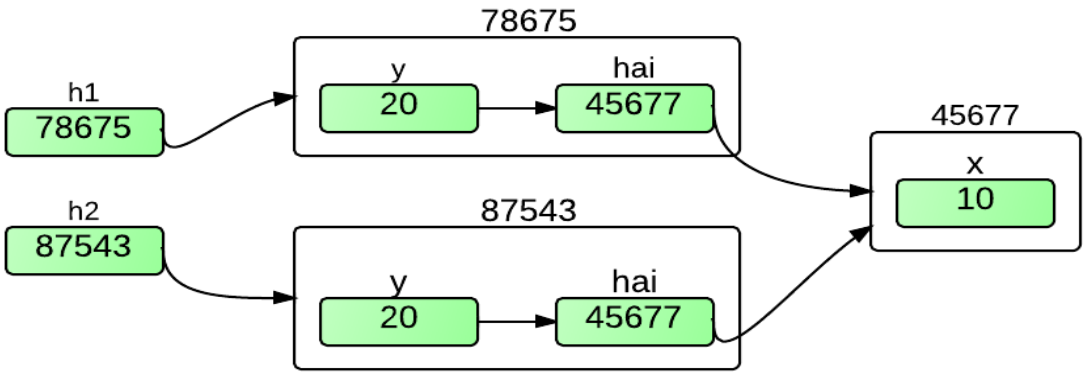
JLANG9.java

```
class Hai{
  int x;
    Hai(int x){
    this.x=x;
  }
}
class Hello implements Cloneable{
int y;
Hai hai;
Hello(int y, Hai hai){
this.y=y;
this.hai=hai;
}
void show(){
System.out.println("Hello->y : "+y);
System.out.println("Hai->x : "+hai.x);
}
public Object clone() throws
          CloneNotSupportedException{
return super.clone();
   }
}
```

```
public class JLANG9 {
public static void main(String[] args)
throws CloneNotSupportedException{

Hai hai=new Hai(10);
Hello h1=new Hello(20,hai);
Hello h2=(Hello)h1.clone();
h1.show();
h2.show();
System.out.println(h1==h2);
System.out.println(h1.hai==h2.hai);
h2.y=30;
h1.show();
h2.show();
h2.hai.x=111;
h1.show();
h2.show();
   }
}
```

**Here:-



2. Deep Cloning

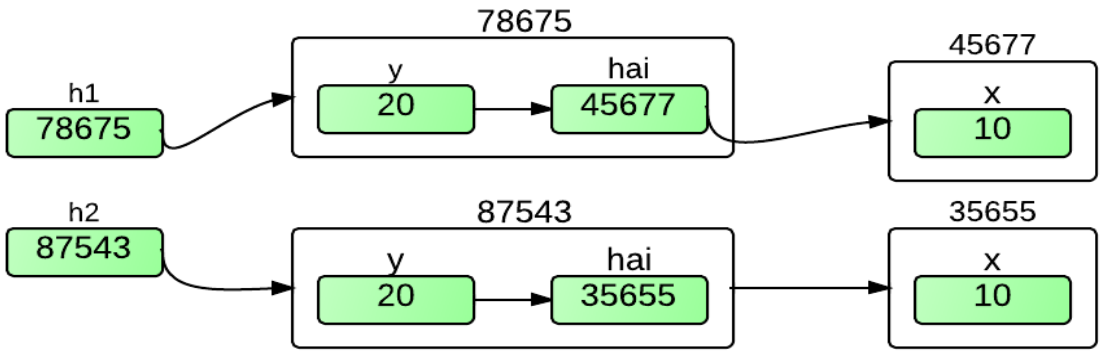
- There is not default implementation for deep cloning.
- You need to define the implementation.
- In this type of cloning the current object (that is used to invoke clone) will be cloned as well as any member object will be closed.
- If you modify the data of cloned object then it won't affect the actual object.
- If you modify the data of the member object then it also won't affect the other objects.

JLANG10.java

```
class Hai{
  int x;
    Hai(int x){
    this.x=x;
  }
}
class Hello implements Cloneable{
int y;
Hai hai;
Hello(int y, Hai hai){
this.y=y;
this.hai=hai;
}
void show(){
System.out.println("Hello->y : "+y);
System.out.println("Hai->x : "+hai.x);
}
public Object clone() throws
          CloneNotSupportedException{
if (this instanceof  Cloneable){
Hai hai=new Hai(this.hai.x);
Hello h=new Hello(this.y,hai);
return h;
}
else{
throw new CloneNotSupportedException
(getClass().getName());
   }
  }
}
public class JLANG10 {
public static void main(String[] args)
throws CloneNotSupportedException{

Hai hai=new Hai(10);
Hello h1=new Hello(20,hai);
Hello h2=(Hello)h1.clone();
h1.show();
h2.show();
System.out.println(h1==h2);
System.out.println(h1.hai==h2.hai);
h2.y=30;
h1.show();
h2.show();
h2.hai.x=111;
h1.show();
h2.show();
   }
}
```

**Here:-



Memory Management

- When you are starting the JVM then JVM will request some memory from the OS. If OS allocates the required memory then JVM will start otherwise the error message will be displayed and JVM will not start.

JLANG11.java	**Execute as follows**

public class JLANG11 {
public static void main(String[] args){
System.*out*.println("Main Started");
Runtime rt=Runtime.*getRuntime*();
System.*out*.println("Tot :" +rt.totalMemory());
System.*out*.println("Mem : "+rt.maxMemory());
}}
	java –Xms 512m JLANG11
	java –Xms 512m JLANG11
	java –Xms 2048m JLANG11
	java –Xms512m –Xmx 1024 JLANG11
	
	-Xms -> Initial Memory
	-Xmx -> Max Memory

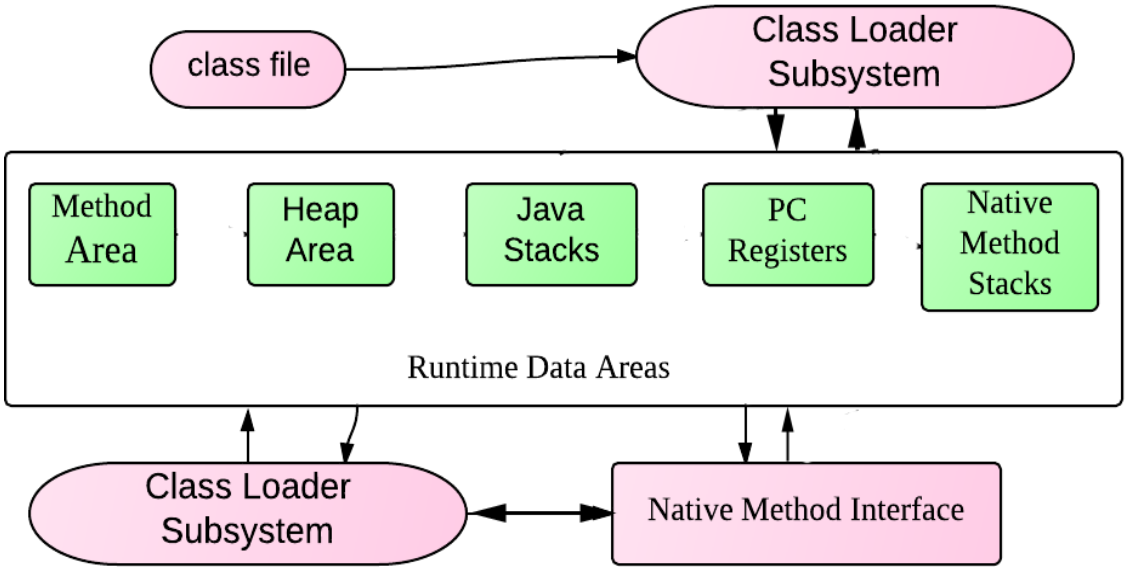
- When a JVM executes a program, it needs memory to store man things like:
  - ⇒ Byte codes and other information extracted from loaded class files.
  - ⇒ Object created in your program.
  - ⇒ Methods Parameters.
  - ⇒ Values returned from methods.
  - ⇒ Local variables declared.
  - ⇒ Intermediate result of computations.
- The java Virtual machine organizes the memory into several runtime data areas.
  - ⇒ Methods Area/Class Heap
  - ⇒ Heap Memory
  - ⇒ Stack Memory

Method Area/Class Heap

- When the class will be loaded b the JVM then the class information will be stored into the method area.
- Static variable also gets the memory in method area.
- Each instance of the Java virtual machine has one method area.
- These areas are shared by all threads running inside the virtual machine.
- These memories should not be allocated during the execution of the application.

Heap Memory

- JVM places all objects into the heap memory.
- Each instance of the JVM has one heap area.
- These areas are shared by all threads running inside the virtual machine.
- If you have the reference of the object in your application then it will be known as used or live object.
- If you do not have the reference for the object in your application then the object will be known as unused or dead object are called as garbage.
- There is no guarantee that memory for the unused or dead objects will be de-allocated immediately.
- When you create the object and required memory is not available in the Heap then OutOfMemoryError will be thrown by JVM.



Stack Memory

- Stack memory keep track of each and every method invocation. This is called Stack Frame.
- Each thread has its own pc register(program counter) and Java stack frame.
- When the method or constructor is invoked then the implementation will be copied into the stack memory on the top of the Stack will be de-allocated.
- Memory for Local variable and method arguments will be allocated in Stack memory.
- The program counter always keeps track of the current instruction which is being executed. After execution of an instruction, the JVM sets the PC to next instruction.
- When you invoke method or constructor and required memory is not available in the Stack then StackOverflowError will be thrown by JVM.

Memory allocation for following

Instance Variable	Heap memory
Static variable	class Heap/Method area
Method	Stack
Method parameter	Stack
Local variable	Stack
Constructor	Stack
Constructor parameter	Stack
instance block	Stack
primitive	As per Scope

Garbage Collector (GC)/Protected void finalize()

- In the case of C, You can use **malloc() or calloc()** functions to allocate the memory and **free()** function to de-allocate that memory.
- In the case of C++, You can use new operator to allocate the memory and delete operator to de-allocate the memory which is allocated by new operator.
- In the case of Java, You can use new operator to allocate the memory but there is no operator or function is designed to de-allocate the memory explicitly.
- In Java, automatic memory cleaning process is designed to reduce the developer burden.
- The process of cleaning the memory of unused or un-referred object by the JVM using GC thread is called as Garbage Collection.
- JVM is responsible to identify the unused object and de-allocates the memory by invoking GC.
- GC is a service thread which is running inside the JVM.

JVM use the following techniques to identify the unused objects.

Case 1:-

- When you are accessing any instance members of class dynamically i.e. without storing the address in reference variable then after accessing that member's object will be eligible for GC.

```
new Hello().m1();
```

Case 2:-

- When you are accessing null to reference variable then the object pointed by reference variable will be unused or un-referred. That object will be eligible for GC.

```
Hello h= new Hello ();
h=null;
```

Case 3:-

- When you are accessing one reference variable to another reference variable the Object pointed by the reference variable which is left side of assignment operator will be unused or un-referred. That object will be eligible for GC.

```
Hello h1= new Hello ();
Hello h2= new Hello ();
h1=h2;
```

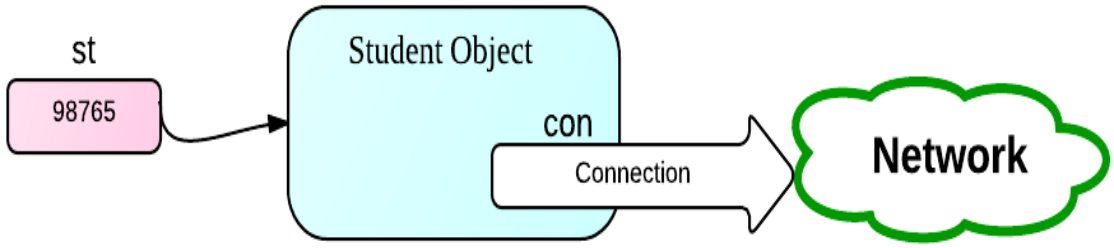
Case 4:-

- When object is out of scope i.e. when you are creating locally either inside methods, blocks or constructors then outside that block object will be eligible for GC.

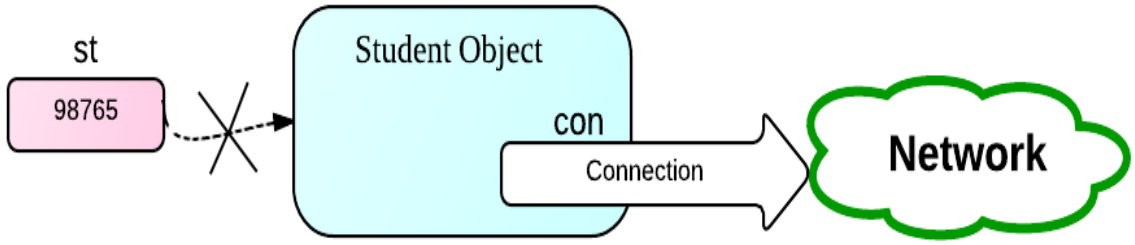
```
Void m1() {
Hello h1= new Hello ();
Hello h2= new Hello ();
Hello h3= new Hello ();
}
```

- Sometimes some objects may refuse to clean the memory because those object still holding some connection like JDBC connection, IO Connections or some network connection.
- If you release those resources then GC can clean the memory without any problems.

Creating the object & connect to network



After using the object you are assign null value to reference.



- JVM will invoke runFinalization() and finalize() method on all unused objects.
- If you want to release the resource then you can override the finalize () method of Object class inside your class and you can write the resource cleanup code.

Process of Garbage Collection

1. List of unused object will be prepared.
2. runFinalization() method will be called to release the resources.
3. GC will be called to clean the memory of unused objects.

- You or JVM can invoke runFinalization() method as

```
System.runFinalizaion();
        (or)
Runtime rt=Runtime.getRuntime();
rt. runFinalization();
```

- You or JVM can invoke gc() method as

```
System.gc();
        (or)
Runtime rt=Runtime.getRuntime();
rt. gc();
```

- Object class finalize() method has no implementations.

```
Protected void finalize( ){ }
```

- You have to override finalize() method in your class to release the resources used in your class.
- You can invoke the finalize () method explicitly like other methods. When you invoke finalize() method explicitly then it will release the resource only(it won't release the object memory).
- Since GC is a thread so we can request JVM to invoke GC, We can't force to invoke GC immediately.

JLANG12.java

```
class Hai{
public void finalize (){
System.out.println("Hai->finalize()");
}
}
class Hello{
void show() {
System.out.println("show");
}
void m1(){
System.out.println("m1-start");
Hai hai1=new Hai();
Hai hai2=new Hai();
Hai hai3=new Hai();
System.out.println("m1-End");
}
public void finalize(){
System.out.println("Hello-finalize");
   }
}
public class JLANG12 {
public static void main(String[] args){
new Hello().show();
Hello h1=new Hello();
h1=null;
Hello h2=new Hello();
Hello h3=new Hello();
h2=h3;
new Hello().m1();
//System.runFinalization();
System.gc();
}
}
```

NOTE: - If the memory space will not be available then JVM will find out the list of unused object. If no unused object will be found then JVM will throw the memory error.

java.lang.OutOfMemoryError.

JLANG13.java

```
public class JLANG13 {
public static void main(String[] args){
Employee emps[]=new Employee[500];
for (int i = 0; i < emps.length; i++) {
//new Employee("Sai"+(i+1));
emps[i]=new Employee("Sai-" +(i+1));
}
}
}
class Employee{
String eid;
double add[]=new double[123456];
Employee(String eid){
this.eid=eid;
System.out.println("Obj Created with id : "+eid);
}
protected void finalize(){
System.out.println("FINALIZE -"+eid);
}
}
```

System.runFinalizersOnExit (boolean invGC)

- This method is deprecated.
- If value of invGC is true then After completing the main() method (but before destroying JVM), GC will be invoked and clean memory occupied by all object.

JLANG14.java

```
public class JLANG14 {
public static void main(String[] args){
Student st=new Student("STU-001");
new Student("STU-002");
System. runFinalizersOnExit(true);
System.out.println("Main completed");
   }
}
class Student{
String sid;
Student( String sid) {
this .sid=sid;
System.out.println("Obj Created with id : "+sid);
}
protected void finalize(){
System.out.println("FINALIZE " +sid);
}
```

String class

- String is a built-in class in java.lang package.
- String is final class, so you can't define the subclass for String class.
- String class implements the following interface.
  - ⇒ java.io.Serializable
  - ⇒ java.lang.Comparable
  - ⇒ java.lang.CharSequence
- String class has following variable to hold data.
  private final char value[];
- Following is the String class definition by Java Vender.

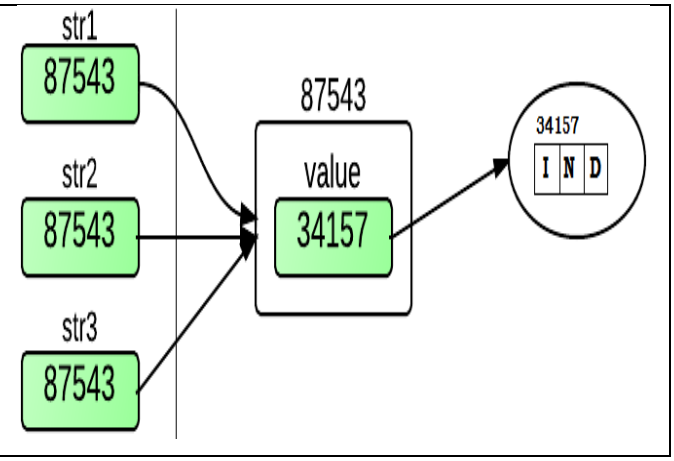
```
public final class String implements java.io.Serializable, Comparable, CharSequence {
  private final char value[];
  .....
}
```

- String object are immutable objects. It means once the object is created then the content or data of the object can't be modified.
- When you try to modify the contents of object then new object will be created as a result.
- You can create the object to String in two ways.
  - ⇒ Without using new operator
  - ⇒ Using new operator

Creating String object without new operator:

JLANG15.java

```
public class JLANG15 {
public static void main(String[] args){
String st1="IND";
String st2="IND";
String st3="IND";
System.out.println(st1);
System.out.println(st2);
System.out.println(st3);
System.out.println(st1==st2);
System.out.println(st1==st3);
  }
}
```

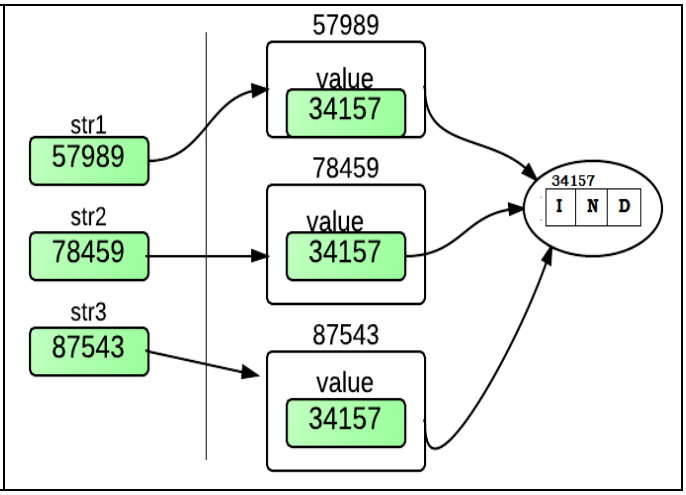


- When the String data is statically (used in the Source code) then all the reference variable will refer same object.
- When the String data is dynamically (getting from keyboard or Command line argument) then always new memory will be allocated in the Heap memory.
- JVM allocate the memory for the string reference variable.
- JVM verifies the String literal in String Constant Pool.
- If String literal is not available in String Constant Pool then following tasks will happen.
  - ⇒ If allocates the memory for value reference variable. (Part of String object)
  - ⇒ Create the character array in String constant pool.
  - ⇒ Assigns the array object address to value reference variable.
  - ⇒ String object address will be assigned to String reference variable.

Creating String object with new operator:

JLANG16.java

```
public class JLANG16 {
public static void main(String[] args){
String st1="IND";
String st2=new String("IND");
String st3=new String("IND");
System.out.println(st1);
System.out.println(st2);
System.out.println(st3);
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st2==st3);
  }
}
```



- JVM allocates the memory for the String reference variable.
- JVM create the String object.
  - ⇒ JVM allocates the memory for value reference variable(Part of String object).
- JVM verifies the String literal in String Constant Pool.
- If String literal is available then existing String Object address will be assigned to String Reference variable.
- If String literal is not available in String Constant Pool then following tasks will happen:
  - ⇒ Create the character array in String constant pool.
  - ⇒ Assigns the array object address to value reference variable.
- If Sting literal is available then following tasks will happen:
  - ⇒ Assigns the array object address to value reference variable.
- String object address will be assigned to String reference variable.
- If you are using String Concatenation operator then
  - ⇒ If both the operands are constant then memory from constant pool will be referred.
  - ⇒ If any of the operand will be non-final variable then the new memory will be allocated in the Heap memory and that reference will be returned.

to get all the String method, Constructor **(javap java.lang.String).**

Important Constructor from String Class	
public String();	Create a new String with the empty content
public String(String str);	Create a new String with the specified content
public String(char[] arr);	Create a new String with the specified array of chars
public String(char[], int stIndex, int len)	Create a new String with the specified part of array of chars
public String(byte[],int stIndex, int len)	Create a new String with the specified part of array of chars
public String(byte[]);	Create a new String with the specified
public String(StringBuffer);	Create a new String with the specified
public String(StringBuilder);	Create a new String with the specified

Important Method from String Class	
public nativeString intern()	Returns the reference variable of string object from Constant pool
public int length()	Returns the length of string object
public boolean isEmpty()	Return true if, and only if, length() is 0.
public String concat(String)	Concatenates the specified string to the end of current string
public String toLowerCase()	Convert all the characters in the end of lower case.
public String toUpperCase()	Convert all the characters in the end of Upper case.
public String trim()	Trim whitespace from the beginning and end of a string.
public static String valueOf (X val) here, X can be Object, int, boolean, double etc	Returns the String representation of the Specified argument.
Public char charAt (int index)	Returns the char value at the specified index.
Public char [] toCharArray ()	Returns the char value at the specified array.
public void getChars(int srcBegin, intsrcEnd, char[] dst, int dstBegine)	Copies character from this string into the destination character array.
public byte[] getbytes()	Converts this string to a new byte array using ASCII.
public void getBytes(srcBegin, intsrcEnd, byte[] dst, int dstBegine)	Copies ASCII of the characters from this string into the destination byte array
public boolean equals (Object)	Compares content of this string to the specified object. Case of the character also will be verified.
public boolean equalsIgnoreCase(String)	Compares content of this string to the specified object. Case of the character also will be ignored.
public boolean contentEquals(StringBuffer)	Compares content of this string to the specified StringBuffer object. Case of the character also will be verified.
public boolean contentEquals(CharSequence)	Compares content of this string to the specified CharSequence object .Case of the character also will be verified.
public int compareTo(String);	Compares content of this string to the specified object Case of the character also will be verified.
public int compareToIgnoreCase(String);	Compares content of this string to the specified object Case of the character also will be not verified.
public boolean startsWith (String prefix)	Checks whether string starts with specified prefix.
public boolean startsWith(String p, int fromIdx))	Checks whether string starts with specified prefix from specified starting index.
public boolean endsWith(String suffix)	Checks whether string starts with the specified suffix.
public int indexOf(int ch)	Returns the index of the first occurrence of the specified character.
public int indexOf(int ch, int frmIndex)	Returns the index of the first occurrence of the specified character. It starts search from specified index.
public int indexOf(String st)	Returns the index of the first occurrence of the specified string.
public int indexOf(String st, int frmIdx)	Returns the index of the first occurrence of the specified String. It starts search from specified index.
public int lastIndexOf(int ch)	Returns the index of the last occurrence of the specified character.

public int lastIndexOf(int ch, int frmIndex)	Returns the index of the last occurrence of the specified character. It starts search backward from specified index.
public int lastIndexOf(String st)	Returns the index of the last occurrence of the specified string.
public int lastIndexOf(String st, int frmIdx)	Returns the index of the last occurrence of the specified String. It starts search backward from specified index.
public String substring(int beginIdx)	Returns the part of String. Substring begine with the character at the specified index and up to the end of this string.
public String substring(int beginIdx, int endIdx)	Returns the part of the String. Substring begins with the character at the specified index and up to the (endIndex-1).
public CharSequence subsequence int beginIdx,int endIdx)	Returns the part of the String as CharSequence. Result SubSubstring begins with the specified index and up to the (endIndex-1).
public String replace(char oChar, char nChar)	Replaces all occurrences of oChar with nChar.
public String replaceFirst(String regEx, String newString);	Replace first occurrences of matching String with newString .
public String replaceAll(String regEx, String newString);	Replace all occurrences of matching String with newString .
public String replace(CharSequence target, CharSequence newCharSeq);	Replace first occurrences of matching Charsequence with newCharsequence .
public boolean matches(String regExp);	Check whether this string matchs with the given regular expression.
public boolean contains (CharSequence);	Compares content of this string to the specified CharSequence.
public boolean regionMatches(int fromIdx, String otherStr, int fromIdxIn, int len	Compares part or regions of two strings. Case of the character also will be verified.
public boolean regionMatches(boolean ignoreCase, intfromIdx, String otherStr, intfromIdxInOtherStr, int len);	Compares part or regions of two strings. When ignoreCase is true then case of the character also will be ignored.
public String[] split(String regEx)	Splits this string when the given regular expression matches.
public String[] split(String regEx, int limit)	Splits this string when the given regular expression matches. Limit controls the number of times the pattern is applied and it affects the length of the resulting of the resulting array.
public static String format(String, Object ...)	Returns a formatted string using the specified string and arguments.
public int hashCode();	Returns the hashCode of String object. Uses contents of String to generate the hashCode.

String class equals ()

- Inherited from Object class and Overridden in String class.
- Return true when only if argument is
  - ⇒ not null
  - ⇒ String object
  - ⇒ Represent the same sequence of characters as this object.

String class compareTo()

Up to java 1.4

- Inherited from Object class (public int compareTo (Object obj).
- New Method in String class (public int compareTo (String obj).
- You can provide any type of argument, if it is String type then will be compared otherwise java.lang.ClassCastException will be thrown.

From java 5

Inherited Method (Because of Generics)

public int compareTo(String obj).

Note: - You cannot provide other than String as argument.

- This Method will compare the String content by using ASCII value.
- If ASCII different will be found then difference will be returned otherwise length difference will be returned.
  - ⇒ If it is returning 0(zero) then both the object will be same.
  - ⇒ If it is returning +ve then first object will be higher or greater.
  - ⇒ If it is returning -ve then second object will be higher or greater.
- To compare the data without verifying the case you can use the following method.

  public int compareToIgnreCase(String obj);

String class valueOf(X val)

- If you are providing reference of object the toString() method wit object will be called and String value will be returned.

String class IndexOf()/lastIndexof()

- Returns -1 if the character or string is not found.

String class substring(int beginIndex, int endIndex)

- The length of the substring is endIndex-beginIndex.

String class split()

- If you are not specifying the sizeOfResultArray then String will be split using all the occurrence of the expression.
- If you are specifying the sizeOfResultArray as 0(zero) or more then occurrence of the expression then String will be split using all the occurrence of the expression.
- If you are not specifying the sizeOfResultArray as more 0(zero) but less then occurrence of the expression then String will split up to that occurrence only.

String class haseCode()

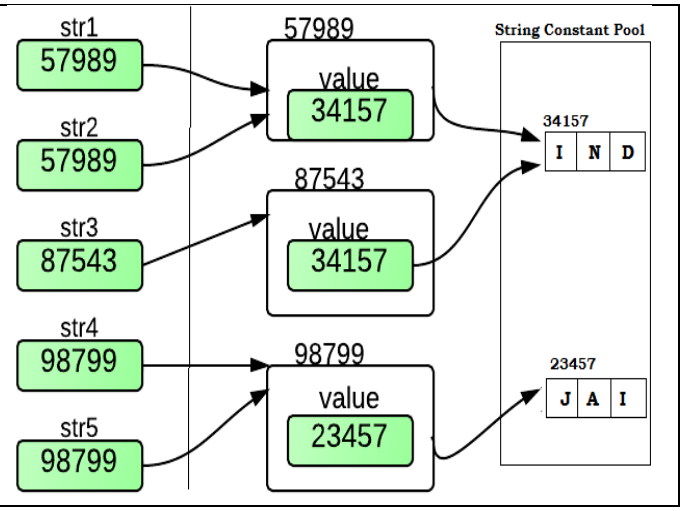
- It used ASCII of the character to generate the hashCode value.
- If two strings is referencing different object but if the content will be same then hashCode will be same but we can't say that both are pointing same object.
- The following implementation for hashCode in String class.

(ASCII of 1^{st} Char*$31^{(len-1)}$ + (ASCII of 2^{st} Char*$31^{(len-2)}$ +(ASCII of 3^{st} Char*$31^{(len-3)}$ +(ASCII +.....+of n^{st} Char*$31^{(len-n)}$

Programs:-

JLANG17.java

```
public class JLANG17 {
public static void main(String[] args){
String st1="IND";
String st2=new String("IND");
String st3=st2.intern();
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st2==st3);
String st4="JAI".intern();
String st5="JAI";
System.out.println(st4==st5);
        }
}
```



JLANG18.java

```
public class JLANG18 {
public static void main(String[] args){
Stud st=new Stud("SAI");
Emp em=new Emp("SAI");
System.out.println(st.snm==em.enm);
st.show(em);
   }
}
class Stud{
          String snm;
          Stud(String snm){
          this.snm=snm;
          }
```

```
void show(Emp em){
     String msg="SAI";
     System.out.println(em.enm==msg);
    System.out.println(em.enm==snm);
    em.display(this);
  }
}
class Emp{
    String enm;
    Emp(String enm){
         this.enm=enm;
     }
void display(Stud st){
         String var="SAI";
        System.out.println(st.snm==var);
        System.out.println(st.snm==enm);
    }
}
```

JLANG19.java

```
import java.util.Scanner;
public class JLANG19 {
public static void main(String[] args){
String st1="IND";
String st2=" IND ";
System.out.println(st1==st2);
Scanner sc=new Scanner(System.in);
System.out.println("Enter String:");
String s3=sc.nextLine();
System.out.println("Enter String:");
String s4=sc.nextLine();
System.out.println(s3==s4);
          }
}
```

JLANG20.java

```
import java.util.Scanner;
public class JLANG20 {
public static void main(String[] args){
String st1="IND";
Scanner sc=new Scanner(System.in);
System.out.println("Enter String:");
String st3=sc.nextLine();
System.out.println("Enter String:");
String st4=sc.nextLine();
String st5=st3.intern();
String st6=st4.intern();
System.out.println(st3==st4);
System.out.println(st5==st6);
System.out.println(st1==st5);
          }
}
```

JLANG21.java

```
public class JLANG21 {
public static void main(String[] args){
String st1="IND";
String st2="IA";
String st3=st1+st2;
String st4=st1+"IA";
String st5="IND"+st2;
System.out.println(st3+"\t"+st4+"\t"+st5);
System.out.println(st3==st4);
System.out.println(st3==st5);
System.out.println(st4==st5);
        }
}
```

JLANG22.java

```
public class JLANG22 {
public static void main(String[] args){
String st1="INDIA";
String st2="IND"+"IA";
final String st3="IND";
final String st4="IA";
String st5=st3+st4;
System.out.println(st1==st2);
System.out.println(st1==st5);
        }
}
```

JLANG23.java

```
public class JLANG23 {
public static void main(String[] args){
String st1="IND101";
String st2="IND";
final int ab=101;
String st3=st2+ab;
System.out.println(st1+"\t"+st2);
System.out.println(st1==st3);
        }
}
```

JLANG24.java

```
public class JLANG24 {
public static void main(String[] args){
String st1=" IND101";
final String st2="IND";
final int ab=101;
String st3=st2+ab;
String st4="IND"+101;
System.out.println(st1+"\t"+st3+"\t"+st4);
System.out.println(st1==st3);
System.out.println(st1==st4);
        }
}
```

JLANG25.java

```
public class JLANG25 {
public static void main(String[] args){
String st1="GTEAT";
String st2="INDIA";
String st3=st1.concat(st2);
System.out.println(st3);
String st4="GREATINDIA";
System.out.println(st3==st4);
        }
}
```

JLANG26.java

```
public class JLANG26 {
public static void main(String[] args){
int x=10;
System.out.println("X="+x=="X"+x);
final int a=10;
System.out.println("A="+a=="A"+a);
        }
}
```

JLANG27.java

```
public class JLANG27 {
public static void main(String[] args){
int ab=98;
int bc=98;
//System.out.println("Res  is:"+ab==bc);
System.out.println("Result is : "+ab);
}
}
```

JLANG28.java

```
public class JLANG28 {
public static void main(String[] args){
int ab=98;
int bc=98;
System.out.println("Result is : "+(ab==bc));
}
}
```

JLANG29.java

```
public class JLANG29 {
public static void main(String[] args){
String st1=new String("IND");
String st2=new String("IND");
String st3=new String("india");
System.out.println(st1+"\t"+st2+"\t"+st3);
System.out.println(st1==st2);
System.out.println(st1==st3);
System.out.println(st1.equals(st2));
System.out.println(st1.equals(st3));
System.out.println(st1.equalsIgnoreCase(st3));
        }
}
```

JLANG30.java

```
public class JLANG30 {
public static void main(String[] args){
System.out.println("ABC".compareTo("ABC"));
System.out.println("ABC".compareTo("ADO"));
System.out.println("ABC".compareTo("ABCDEF
GH"));
System.out.println("ABCDEFGH".compareTo("D
EF"));
System.out.println("ABC".compareTo("abc"));
System.out.println("ABC".compareToIgnoreCase
("abc"));
        }
}
```

JLANG31.java

```
public class JLANG31 {
public static void main(String[] args){
int ab=98;
String st1=ab;
String st2=(String) ab;
System.out.println(st1+"\t"+st2);
        }
}
```

JLANG32.java

```
public class JLANG32 {
public static void main(String[] args){
int ab=98;
String st1=String.valueOf (ab);
String st2=String.valueOf (true);
System.out.println(st1+"\t"+st2);
   }
}
```

JLANG33.java

```
public class JLANG33 {
public static void main(String[] args){
Student std=new Student();
String st1=String.valueOf(std);
System.out.println(st1);
Employee emp=new Employee();
String st2=String.valueOf(emp);
System.out.println(st2);
    }
}
class Student{}
class Employee{
int eid;
public String toString(){
return "Eid:"+eid;
    }
}
```

JLANG34.java

```
public class JLANG34 {
public static void main(String[] args){
Student stu=null;
String st1=String.valueOf(stu);
System.out.println(st1);
        }
}
```

.....

JLANG35.java

```
public class JLANG35 {
public static void main(String[] args){
String st1=String.valueOf(null);
System.out.println(st1);
        }
}
```

JLANG36.java

```
public class JLANG36 {
public static void main(String[] args){
String st1="INDIA";
int len=st1.length();
System.out.println(st1+"\t"+len);
String st2=st1.toLowerCase();
String st3=st1.toUpperCase();
System.out.println(st2);
System.out.println(st3);
        }
}
```

JLANG37.java

```
public class JLANG37 {
public static void main(String[] args){
String st1="INDIA";
String st2=st1.trim();
System.out.println(st1+"\t"+st2);
System.out.println(st1==st2);
String st3="  INDIA  ";
String st4=st3.trim();
System.out.println(st3+"\t"+st3.length());
System.out.println(st4+" \t"+st4.length());
System.out.println(st3==st4);
}
}
```

JLANG38.java

```
public class JLANG38 {
public static void main(String[] args){
String st1="";
System.out.println(st1.length());
System.out.println(st1.isEmpty());
        }
}
```

JLANG39.java

```
public class JLANG39 {
public static void main(String[] args){
String st1=null;
System.out.println(st1.length());
        }
}
```

JLANG40.java

```
public class JLANG40 {
public static void main(String[] args){
String st1=null;
System.out.println(st1.isEmpty());
        }
}
```

JLANG41.java

```
public class JLANG41 {
public static void main(String[] args){
System.out.println(null. length());
        }
}
```

JLANG42.java

```
public class JLANG42 {
public static void main(String[] args){
String st1="Hai, Welcome to java";
System.out.println(st1.startsWith("Hai"));
System.out.println(st1.startsWith(
"Welcome"));
System.out.println(st1.startsWith(
"Welcome", 5));
        }
}
```

JLANG43.java

```
public class JLANG43 {
public static void main(String[] args){
String st1="Hai,welcome to java";
System.out.println(st1.endsWith("java"));
System.out.println(st1.endsWith("Welcome"
));
System.out.println(st1.startsWith(""));
System.out.println(st1.endsWith(""));
        }
}
```

JLANG44.java

```
public class JLANG44 {
public static void main(String[] args){
String st1="Hai,Welcome to India(India is
great),great India";
String st2=st1.replace('I', 'x');
System.out.println(st2);
String
st3=st1.replaceFirst("great","Delhi");
System.out.println(st3);
String st4=st1.replaceAll("Indai","Delhi");
System.out.println(st4);
System.out.println(st1);
}}
```

JLANG45.java

```
public class JLANG45 {
public static void main(String[] args){
String st1="Hai,Welcome to India, the great
India";
System.out.println(st1.indexOf(97));
System.out.println(st1.indexOf('a'));
  }
}
```

JLANG46.java

```
public class JLANG46 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is great, Delhi is capital of Indai";
System.out.println(st1.indexOf('I'));
System.out.println(st1.indexOf('I',11));
System.out.println(st1.indexOf('I',12));
System.out.println(st1.indexOf('I',17));
        }
}
```

JLANG47.java

```
public class JLANG47 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is great, Delhi is capital of India";
System.out.println(st1.indexOf("INDIA"));
System.out.println(st1.indexOf("INDIA",16));
System.out.println(st1.indexOf("INDIA",17));
        }
}
```

JLANG48.java

```
public class JLANG48 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is great, Delhi is capital of India";
System.out.println(st1.indexOf(st1));
System.out.println(st1.lastIndexOf(97));
System.out.println(st1.lastIndexOf('a'));
   }
}
```

JLANG49.java

```
public class JLANG46 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is great, Delhi is capital of INDIA";
System.out.println(st1.lastIndexOf("INDIA"));
System.out.println(st1.lastIndexOf("INDIA",46));
System.out.println(st1.lastIndexOf("INDIA",45));
   }
}
```

JLANG50.java

```
public class JLANG50 {
public static void main(String[] args){
String st1="GREAT INDIA";
System.out.println(st1.substring(0));
System.out.println(st1.substring(3));
System.out.println(st1.substring(0,4));
System.out.println(st1.substring(3,5));
        }
}
```

JLANG51.java

```
public class JLANG51 {
public static void main(String[] args){
String st1="GREAT INDIA";
System.out.println(st1.substring(3,9));
        }
}
```

JLANG52.java

```
public class JLANG52 {
public static void main(String[] args){
String st1="GREAT INDIA";
int len=st1.length();
System.out.println(st1.substring(3,len));
System.out.println(st1.substring(3,len-1));
System.out.println(st1.substring(3,len-2));
        }
}
```

JLANG53.java

```
public class JLANG53 {
public static void main(String[] args){
String st1="INDIA";
int len=st1.length();
System.out.println(st1);
System.out.println(st1.charAt(0));
System.out.println(st1.charAt(3));
System.out.println(st1.charAt(4));
for(int i=0;i<len;i++) {
System.out.println(i+"\t"+st1.charAt(i));
    }
  }
}
```

JLANG54.java

```
public class JLANG54 {
public static void main(String[] args){
String st1="INDIA";
int len=st1.length();
char cArr[]=new char[len];
for(int i=0;i<len;i++){
cArr[i]=st1.charAt(i);
}
System.out.println("\n** char array **");
for(int i=0;i<cArr.length;i++){
char ch=cArr[i];
System.out.println("\t"+i+"\t"+ch);
   }
}}
```

JLANG55.java

```
public class JLANG55 {
public static void main(String[] args){
String st1="INDIA";
int len=st1.length();
byte bArr[]=new byte[len];
for(int i=0;i<len;i++){
bArr[i]=(byte)(st1.charAt(i));
}
System.out.println("\n** byte array **");
for(int i=0;i<bArr.length;i++){
byte b=bArr[i];
System.out.println("\t"+b+"\t"+(char)b);
}
}}
```

JLANG56.java

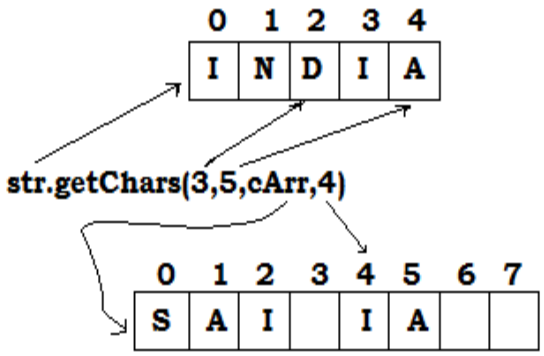
```
public class JLANG56 {
public static void main(String[] args){
String st1="INDIA";
char cArr[]=st1.toCharArray();
System.out.println("\n** char array **");
for(int i=0;i<cArr.length;i++){
char ch=cArr[i];
System.out.println("\t"+i+"\t"+ch);
}}
}
```

JLANG57.java

```
public class JLANG57 {
public static void main(String[] args){
String st1="INDIA";
byte bArr[]=st1.getBytes();
System.out.println("\n** Byte array **");
for(int i=0;i<bArr.length;i++){
byte b=bArr[i];
System.out.println("\t"+b+"\t"+(char)b);
}}
}
```

JLANG58.java

```
public class JLANG58 {
public static void main(String[] args){
String st1="INDIA";
char cArr[]=new char[8];
byte bArr[]=new byte[8];
cArr[0]='S';
cArr[1]='A';
cArr[2]='I';
st1.getChars(3,5,  cArr, 4);
st1. getBytes(3,5,  bArr, 4);
System.out.println("\n** Char array **");
for(int i=0;i<cArr.length;i++){
char ch=cArr[i];
System.out.println("\t"+i+"\t"+ch);
}
System.out.println("\n** Byte array **");
for(int i=0;i<bArr.length;i++){
byte b=bArr[i];
System.out.println("\t"+b+"\t"+(char)b);
      }
    }
}
```



JLANG59.java

```
public class JLANG59 {
public static void main(String[] args){
char cArr[]={'I','N','D','I','A'};
String st="";
for(int i=0; i<cArr.length;i++){
char ch=cArr[i];
st+=ch;
}
System.out.println("from char Array : "+st);
  }
}
```

JLANG60.java

```
public class JLANG560 {
public static void main(String[] args){
byte bArr[]={65,66,67,68,69};
String st="";
for(int i=0; i<bArr.length;i++){
byte b=bArr[i];
st+=b;
}
System.out.println("From byte Array : "+st);
String st1="";
for(int i=0; i<bArr.length;i++){
byte b=bArr[i];
st1+=(char)b;
}
System.out.println("From byte Array : "+st1);
}
}
```

JLANG61.java

```
public class JLANG61 {
public static void main(String[] args){
char cArr[]={'I','N','D','I','A'};
byte bArr[]={65,66,67,68,69};
String st=new String(cArr);
String st1=new String(bArr);
System.out.println("st : "+st);
System.out.println("st1 : "+st1);
  }}
```

JLANG62.java

```
public class JLANG62 {
public static void main(String[] args){
char cArr[]={'I','N','D','I','A'};
byte bArr[]={65,66,67,68,69};
String st1=new String(cArr, 2, 3);
String st2=new String(bArr, 2, 3);
System.out.println("st1 : "+st1);
System.out.println("st2 : "+st2);
}}
```

JLANG63.java

```
public class JLANG63 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is
great, Delhi is capital of INDIA";
String res[]=st1.split("INDIA");
System.out.println("Length :
"+res.length);
for (int i = 0; i < res.length; i++) {
String st=res[i];
System.out.println(i+"\t"+st);
} }}
```

JLANG64.java

```
public class JLANG64 {
public static void main(String[] args){
String st1="Welcome to INDIA. INDIA is
great. Delhi is capital of INDIA";
String res[]=st1.split("\\.");
System.out.println("Lenght : "+res.length);
for (int i = 0; i < res.length; i++) {
System.out.println(i+"\t"+res[i]);
   }
  }
}
```

JLANG65.java

```
public class JLANG65 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is
great, Delhi is capital of INDIA";
String res[]=st1.split("INDIA",0);
System.out.println("Lenght :
"+res.length);
for (int i = 0; i < res.length; i++) {
String st=res[i];
System.out.println(i+"\t"+st);
}
System.out.println("**************");
String res1[]=st1.split("INDIA",5);
System.out.println("Lenght :
"+res1.length);
for (int i = 0; i < res1.length; i++) {
String st2=res1[i];
System.out.println(i+"\t"+st2);
   } }}
```

JLANG66.java

```
public class JLANG66 {
public static void main(String[] args){
String st1="Welcome to INDIA, INDIA is
great, Delhi is capital of INDIA";
String res[]=st1.split("INDIA",1);
System.out.println("Lenght : "+res.length);
for (int i = 0; i < res.length; i++) {
String st=res[i];
System.out.println(i+"\t"+st);
}
System.out.println("**************");
String res1[]=st1.split("INDIA",2);
System.out.println("Lenght : "+res1.length);
for (int i = 0; i < res1.length; i++) {
String st2=res1[i];
System.out.println(i+"\t"+st2);
  }
 }
}
```

JLANG67.java

```
public class JLANG67 {
public static void main(String[] args){
String dir="D:\\eclipsepro\\javalang\\
Java1067\\src";
String res[]=dir.split("\\");
for (int i = 0; i < res.length; i++) {
String st1=res[i];
System.out.println(i+"\t"+st1);
    }
  }
}
```

JLANG68.java

```
public class JLANG68 {
public static void main(String[] args){

String  dir="D:\\eclipsepro\\javalang\\ src";
String res[]=dir.split("\\\\");
for (int i = 0; i < res.length; i++) {
String st1=res[i];
System.out.println(i+"\t"+st1);
    }
  }
}
```

JLANG69.java

```
public class JLANG69 {
public static void main(String[] args){
String exp="[A-Z]";
System.out.println("H".matches(exp));
System.out.println("S".matches(exp));
System.out.println("HI".matches(exp));
  }
}
```

JLANG70.java

```
public class JLANG70 {
public static void main(String[] args){
String exp="[A-Z]*";
System.out.println("H".matches(exp));
System.out.println("IND".matches(exp));
System.out.println("ind".matches(exp));
  }
}
```

JLANG71.java

```
public class JLANG71 {
public static void main(String[] args){
String exp="[A-Za-z0-9]*";
System.out.println("ind99".matches(exp));
System.out.println("Ind99".matches(exp));
System.out.println("99Ind".matches(exp));
}
}
```

JLANG72.java

```
public class JLANG72 {
public static void main(String[] args){
String exp="^[A-Z][A-Za-z0-9]*";
System.out.println("ind99".matches(exp));
System.out.println("Ind99".matches(exp));
System.out.println("99Ind".matches(exp));
}
}
```

JLANG73.java

```
public class JLANG73 {
public static void main(String[] args){
String st1="website is www.google.com";
String st2="java@google.com is id to sent
mail";
boolean b1=st1.regionMatches(2, st2, 5, 10);
System.out.println(b1);
boolean b2=st1.regionMatches(15, st2, 5, 9);
System.out.println(b2);
boolean b3=st1.regionMatches(2, st2, 5, 12);
System.out.println(b3);
boolean b4=st1.regionMatches(15, st2, 5, 13);
System.out.println(b4);
}
}
```

JLANG74.java

```
public class JLANG74 {
public static void main(String[] args){
String st1="AB";
String st2=new String("AB");
System.out.println(st1.hashCode());
System.out.println(st2.hashCode());
System.out.println(st1==st2);
  }
}
```

JLANG75.java

```
public class JLANG75 {
public static void main(String[] args){
String name[]={"India", "Delhi", "Bangalore",
"Agra", "Patna" };
for(int i=0; i<name.length; i++){
for (int j = i+1; j < name.length; j++) {
String st1=name[i];
String st2=name[i];
int res=st1.compareTo(st2);
if(res>0){
name[i]=st1;
name[i]=st2;
    }
  }
}
System.out.println("\n NAMES IN SORTED
ORDER");
for(int i=0; i<name.length; i++){
String nm=name[i];
System.out.println(nm);
     }
   }
}
```

JLANG76.java

```
public class JLANG76 {
public static void main(String[] args){
String str="Welcome to INDIA, INDIA is
great, Delhi is capital of India";
  int len=str.length();
  int count=0;
for (int i=0; i<len; i++){
int indx=str.indexOf("INDIA",i);
if(indx>=0){
i=indx;
count++;
   }
 }
System.out.println("Count :"+count);
    }
 }
```

Program Output

Prog No.	Output	Prog No.	Output
JLANG15	IND IND IND true true	JLANG16	IND IND IND false false false
JLANG17	false true false true	JLANG18	true true true true true
JLANG19	true Enter String: OK Enter String: OK false	JLANG20	Enter String: IND Enter String: IND false true true
JLANG21	INDIA INDIA INDIA false false false	JLANG22	true true
JLANG23	IND101 IND false	JLANG24	IND101 IND101 IND101 true true
JLANG25	GREAT INDIA false	JLANG26	false false
JLANG27	Result is:98	JLANG28	Result is: true

Prog No.	Output	Prog No.	Output
JLANG29	INDIA INDIA india false false true false true	JLANG30	0 -2 -5 -3 -32 0
JLANG31	Compilation Error	JLANG32	Student@c3c749 Eid: 0
JLANG33	98 true	JLANG34	Compilation Error
JLANG35	NullPointerException	JLANG36	INDIA 5 india INDIA
JLANG37	INDIA INDIA true INDIA 10 INDIA 5 false	JLANG38	0 true
JLANG39	NullPointerException	JLANG40	NullPointerException
JLANG41	NullPointerException	JLANG42	true false true
JLANG43	true false true true	JLANG44	Hai,Welocme to xndia(xndia is great), great xndai Hai,Welocme to India(India is Delhi), great Indai Hai,Welocme to India(India is great), great Delhi Hai,Welocme to India(India is great), great Indai
JLANG45	1 1	JLANG46	1
JLANG47	11 18 18	JLANG48	0 57 57
JLANG49	54 18 18	JLANG50	GREAT INDIA AT INDIA GREA AT
JLANG51	AT IND	JLANG52	AT INDIA AT INDI AT IND
JLANG53	INDIA I I A 0 I 1 N 2 D 3 I 4 A	JLANG54	** char array ** 0 I 1 N 2 D 3 I 4 A

Prog No.	Output	Prog No.	Output
JLANG55	** byte array ** 73 I 78 N 68 D 73 I 65 A	JLANG56	** char array ** 73 I 78 N 68 D 73 I 65 A
JLANG57	** Byte array ** 73 I 78 N 68 D 73 I 65 A	JLANG58	*Char array* 0 S 1 I 2 A 3 - 4 I 5 A 6 - 7 - *Bytes array* 0 - 0 - 0 - 0 - 73 I 65 A 0 - 0 -
JLANG59	from char Array : INDIA	JLANG60	From byte Array : 6566676869 From byte Array : ABCDE
JLANG61	st : INDIA st1 : ABCDE	JLANG62	st1 : DIA st2 : CDE
JLANG63	Length : 3 0 Welcome to 1 , 2 is great, Delhi is capital of	JLANG64	Lenght : 3 0 Welcome to INDIA 1 INDIA is great 2 Delhi is capital of INDIA
JLANG65	Lenght : 3 0 Welcome to 1 , 2 is great, Delhi is capital of ********** Lenght : 4 0 Welcome to 1 , 2 is great, Delhi is capital of 3	JLANG66	Lenght : 1 0 Welcome to INDIA, INDIA is great, Delhi is capital of INDIA ********** Lenght : 2 0 Welcome to 1 , INDIA is great, Delhi is capital of INDIA
JLANG67	compilation error (due to \\)	JLANG68	0 D: 1 eclipsepro 2 javalang 3 src
JLANG69	true true false	JLANG70	true true false
JLANG71	true true true	JLANG72	false true false
JLANG73	false true false false	JLANG74	2081 2081 False
JLANG75	NAMES IN SORTED ORDER India Delhi Bangalore Agra Patna	JLANG76	Count :2

1.3. StringBuffer class

- StringBuffer is a final class in java.lang package.
- It will be used to store the sequence of characters.
- This is used when there is a necessary to make a lot of modifications to characters of String.
- StringBuffer is a mutable sequence of characters; it means the contents of the StringBuffer can be modified after creation.
- Every StringBuffer object has a capacity associated with it.
- The capacity of StringBuffer is the number of characters it can hold.
- The capacity increase automatically as more contents added to it.
- The method available in the StringBuffer class is synchronized.
- StringBuffer class is thread-safe i.e. multiple threads cannot access it simultaneously.
- So to solve this problem Java vender has added a new class StringBuilder in Java5.

1.4. StringBuilder class

- StringBuilder is a final class in java.lang package. (Java 5)
- StringBuilder is class functionality is same as StringBuffer only except the methods available in the StringBuilder class are not synchronized.
- StringBuilder class is not thread-safe i.e. multiple threads can access it simultaneously.

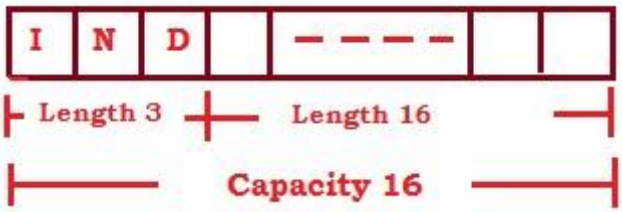
Important methods of StringBuilder	
Methods	Description
int capacity()	Returns the capacity of StringBuilder object
int length()	Returns the length of StringBuilder object
void ensureCapacity(int cap)	Ensure the capacity of the StringBuffer Object The new capacity of the StringBuilder is the maximum of, ⇒ The initialCapacity argument passed and (Old capacity *2)+2
void setLength(int len)	Change the length of StringBuilder ⇒ If new length is more than current length then default value of char (ASCII 0) will be added.
void trimToSize()	Reduces the capacity to length
SrtingBuilder append(X value)	Adds the data in the StringBuilder object ⇒ Method is overloaded ⇒ X can be any primitive type/String/Object
StringBuilder insert(int index,X value)	Inserts the data at specified index ⇒ Method is overloaded ⇒ X can be any primitive type/String/Object
StringBuilder deleteCharAt(int index)	Delete the char from the specified index.
StringBuilder delete (int index1,int index2)	Delete the character from the specified range
void setCharAt(int index,char newChar)	Replaces char at the specific index
StringBuilder replace(int idx1,int idx2,String newValue)	Replaces the chars of Specific range with new String
StringBuilder reverse()	Reverse the content of StringBuilder.
etc	

Important Constructor of StringBuilder	
Methods	Description
public StringBuilder()	Create the StringBuilder object with Initial capacity 16.
public StringBuilder(String content)	Create the StringBuilder object with Initial capacity as (length of specified content+16)
public StringBuilder(int capacity)	Create the StringBuilder object with Initial capacity specified (This constructor may throw java.lang.NegativeArraySizeException if the specified capacity is less than 0)

- equal () is not overridden is StringBuilder and StringBuffer class.
- When you want to compare content of StringBuilder or StringBuffer object then you need to convert that object into String.
- hashCode() is not overridden in the StringBuilder and StringBuffer class. It uses the default implementation of Object class.
- StringBuilder and StringBuffer are not implementing java.lang.ComparableInterface, so You cannot use compareTo() with StringBuilder and StringBuffer.

StringBuilder sb=new StringBuilder();

StringBuilder sb=new StringBuilder("**IND**");



StringBuilder sb=new StringBuilder(25)

Programs:

JLANG77.java

```
public class JLANG77 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder();
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());

StringBuilder sb2=new StringBuilder("IND");
System.out.println("C : "+ sb2.capacity());
System.out.println("L : " +sb2.length());

StringBuilder sb3=new StringBuilder(25);
System.out.println("C : " +sb3.capacity());
System.out.println("L : " +sb3.length());
}
}
```

JLANG78.java

```
public class JLANG78 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder(-3);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG79.java

```
public class JLANG79 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder(9988776655);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG80.java

```
public class JLANG80 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder("IND");
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
sb1.append("  India is great");
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG81.java

```
public class JLANG81 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("IND");
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
sb.append(123.45);
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
sb.append( true);
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG82.java

```
public class JLANG82 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder("IND");
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
sb.setLength(3);
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG83.java

```
public class JLANG83 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder("IND");
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
sb.setLength(15);
System.out.println(sb);
System.out.println("C : " +sb.capacity());
System.out.println("L : " +sb.length());
}
}
```

JLANG84.java

```
public class JLANG84 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("OURINDIA");
System.out.println(sb);
sb.setCharAt(2,'X');
System.out.println(sb);
sb.replace(3, 6, "GREAT");
System.out.println(sb);
sb.reverse();
System.out.println(sb);
}
}
```

JLANG85.java

```
public class JLANG85 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("OURINDIA");
System.out.println(sb);
sb.insert(3,true);
System.out.println(sb);
sb.deleteCharAt(2);
System.out.println(sb);
sb.delete(3,8);
System.out.println(sb);
}
}
```

JLANG86.java

```
public class JLANG86 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("OURINDIA");
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
sb.ensureCapacity(12);
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
sb.ensureCapacity(59);
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
}
}
```

JLANG87.java

```
public class JLANG87 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("OURINDIA");
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
sb.ensureCapacity(-2);
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
}
}
```

JLANG88.java

```
public class JLANG88 {
public static void main(String[] args){
StringBuilder sb=new
StringBuilder("OURINDIA");
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
sb.ensureCapacity(8987766555);
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
}
}
```

JLANG89.java

```
public class JLANG89 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder(null);
        System.out.println(sb);
   }
}
```

JLANG90.java

```
public class JLANG90 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder();
sb.append(null);
System.out.println(sb);
}
}
```

JLANG91.java

```
public class JLANG91 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder("IND");
sb.insert(2,null);
System.out.println(sb);
}
}
```

JLANG92.java

```
public class JLANG92 {
public static void main(String[] args){
StringBuilder sb=new StringBuilder("IND");
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
sb.trimToSize();
System.out.println("C : "+sb.capacity());
System.out.println("L : "+sb.length());
}
}
```

JLANG93.java

```
public class JLANG93 {
public static void main(String[] args){
String str="ABC";
StringBuilder sb=new StringBuilder("ABC");
System.out.println(str.equals(sb));
}
}
```

JLANG94.java

```
public class JLANG94 {
public static void main(String[] args){
String str="ABC";
StringBuilder sb=new StringBuilder("ABC");
System.out.println(str.contentEquals(sb));
}
}
```

JLANG95.java

```
public class JLANG95 {
public static void main(String[] args){
String str="ABC";
StringBuilder sb=new StringBuilder("ABC");
System.out.println(str.hashCode());
System.out.println(sb.hashCode());
}}
```

JLANG96.java

```
public class JLANG96 {
public static void main(String[] args){
StringBuilder sb1=new StringBuilder("ABC");
StringBuilder sb2=new StringBuilder("ABC");
System.out.println(sb1.equals(sb2));
}
}
```

JLANG37.java

```
public class JLANG37 {
public static void main(String[] args){
StringBuilder sb1=new StringBuilder("ABC");
StringBuilder sb2=new StringBuilder("ABC");
String str1=sb1.toString();
String str2=sb2.toString();
System.out.println(str1.equals(str2));
}
}
```

String Class	StringBuffer Class
String Objects are immutable.	StringBuffer Objects are mutable.
We can create the object of string class using new operator or without new operator.	We need to use new operator to create the object of StringBuffer class.
String operations such as append would be less efficient if performed using String.	String operations such as append would be less efficient if performed using StringBuffer.
Methods are not synchronized. (it's object is not thread safe)	Methods are synchronized. (it's object is thread safe)
Implementing java.lang.Comparable interface.	Not Implementing java.lang.Comparable interface.
equals () is overridden to compare the Content.	equals () is not overridden to compare the Content.
Concatenation operator (+) can be used.	Concatenation operator (+) can't be used.
hash code () is overridden using content of String.	hash code () is not overridden

Program Output

Prog No.	Output	Prog No.	Output
JLANG77	C : 16 L : 0 C : 19 L : 3 C : 25 L : 0	JLANG78	java.lang.NegativeArraySize Exception
JLANG79	Compilation Error [The literal 9988776655 of type int is out of range]	JLANG80	IND C : 19 L : 3 IND India is great C : 19 L : 19
JLANG81	IND C : 19 L : 3 IND123.45 C : 19 L : 9 IND123.45true C : 19 L : 13	JLANG82	IND C : 19 L : 3 IND C : 19 L : 3
JLANG83	IND C : 19 L : 3 IND--------- C : 19 L : 15	JLANG84	OURINDIA OUXINDIA OUXGREATIA AITAERGXUO
JLANG85	OURINDIA OURtrueINDIA OUtrueINDIA OUtDIA	JLANG86	C : 24 L : 8 C : 24 L : 8 C : 59 L : 8
JLANG87	C : 24 L : 8 C : 24 L : 8	JLANG88	C : 24 L : 8 OutOfMemoryError: Java heap space
JLANG89	java.lang.NullPointerException	JLANG90	The method insert(int, Object) is ambiguous for the type StringBuilder
JLANG91	The method insert(int, Object) is ambiguous for the type StringBuilder	JLANG92	C : 19 L : 3 C : 3 L : 3
JLANG93	false	JLANG94	True
JLANG95	64578 25860399	JLANG96	False
JLANG97	true		